

The Effect of Poor Source Code Lexicon and Readability on Developer’s Cognitive Load

Nikhil Anand

March 28, 2026

The paper investigates how the quality of source code lexicon and readability influences developers’ cognitive load during program comprehension. Program comprehension is a fundamental activity in software development, and a significant portion of development effort is spent understanding existing code before making changes. The authors argue that identifiers (e.g., variable and method names) and comments serve as critical carriers of domain knowledge, and their quality directly affects how easily developers can interpret code. While prior work has established links between identifier quality and overall software quality, there remains limited empirical evidence on how poor lexicon specifically impacts developers’ mental effort during comprehension tasks .

To address this gap, the study explores the relationship between linguistic and structural properties of code and the cognitive load experienced by developers. Using techniques such as functional Near Infrared Spectroscopy (fNIRS) and eye tracking, the authors measure cognitive effort at a fine-grained level while participants perform comprehension and bug localization tasks. The work focuses particularly on “linguistic antipatterns”—poor naming and documentation practices—and examines how these, along with readability issues, affect developers’ ability to process code. Overall, the paper aims to provide empirical evidence that poor source code lexicon and readability increase cognitive load, thereby negatively impacting program comprehension and developer efficiency .

1 Motivation

A substantial portion of software development effort is devoted to understanding existing code before making modifications, making program comprehension a central and costly activity in the software lifecycle. Prior research highlights that identifiers and comments act as key vehicles for embedding domain knowledge within code, enabling developers to reason about functionality and intent. Empirical studies have shown that the quality of identifiers correlates with overall software quality, suggesting that poor naming and documentation practices may hinder comprehension and increase maintenance effort [1].

Despite these insights, there is limited empirical evidence directly linking the quality of source code lexicon—particularly naming inconsistencies and poorly chosen identifiers—to developers’ cognitive load during comprehension tasks. While software engineering research has extensively explored readability and structural metrics, the cognitive processes underlying how developers interpret code remain underexplored. Advances in cognitive neuroscience and program comprehension research have introduced techniques such as fMRI and eye tracking to study developers’ mental processes, yet these approaches are often expensive or lack granularity [2, 3]. This creates a clear need for empirical, fine-grained investigation into how linguistic aspects of code influence cognitive effort, motivating the study.

1.1 Background

The paper builds on the concept of linguistic antipatterns (LAs), which are recurring poor practices in naming, documentation, and identifier usage that negatively affect code understandability. These antipatterns include inconsistencies between method names and behavior, misleading identifiers, and contradictions between comments and implementation. Such issues can introduce confusion, forcing

developers to spend additional cognitive effort reconciling discrepancies between code and its intended meaning [4].

In addition to linguistic factors, prior work has examined structural and readability metrics—such as cyclomatic complexity, lines of code, nesting depth, and adherence to coding conventions—as indicators of code comprehensibility. Studies have shown that these metrics correlate with how easily developers can understand and maintain code, with poorly structured code increasing comprehension difficulty [5]. Furthermore, research in cognitive measurement has explored the use of physiological signals to study program comprehension. Techniques such as fMRI have been used to identify brain regions involved in code understanding, while eye tracking has provided insights into developers’ visual attention patterns during reading tasks [2, 6].

More recently, functional Near Infrared Spectroscopy (fNIRS) has emerged as a practical alternative for measuring cognitive load, offering a non-invasive and portable way to capture brain activity during real-world programming tasks. Unlike traditional methods, fNIRS enables fine-grained analysis of cognitive effort while maintaining ecological validity, making it suitable for studying how specific code features—such as lexicon quality and readability—affect developers’ mental workload.

References

- [1] Florian Deissenboeck and Markus Pizka. “Concise and Consistent Naming”. In: *Proceedings of the 13th IEEE International Workshop on Program Comprehension*. 2006.
- [2] Janet Siegmund et al. “Understanding Understanding Source Code with Functional Magnetic Resonance Imaging”. In: *Proceedings of the 36th International Conference on Software Engineering*. 2014.
- [3] Benjamin Floyd et al. “Decoding the Representation of Code in the Brain”. In: *Proceedings of the 2017 IEEE/ACM International Conference on Software Engineering*. 2017.
- [4] Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. “Linguistic Antipatterns: What They Are and How Developers Perceive Them”. In: *Proceedings of the 2013 IEEE International Conference on Software Maintenance*. 2013.
- [5] Raymond P. L. Buse and Westley Weimer. “Learning a Metric for Code Readability”. In: *Proceedings of the 2010 IEEE International Conference on Software Maintenance*. 2010.
- [6] Thomas Fritz et al. “Using Psycho-Physiological Measures to Assess Task Difficulty in Software Development”. In: *Proceedings of the 36th International Conference on Software Engineering*. 2014.